

TECHNICAL DOCUMENTATION

RAG-Based Customer Support Assistant

Document Type	Technical Documentation
Project	RAG Customer Support System with LangGraph & HITL
Version	1.0
Status	Final

TABLE OF CONTENTS

1. Introduction
2. System Architecture Explanation
3. Design Decisions
4. Workflow Explanation
5. Conditional Logic
6. HITL Implementation
7. Challenges & Trade-offs
8. Testing Strategy
9. Future Enhancements

1. INTRODUCTION

1.1 What is RAG?

Retrieval-Augmented Generation (RAG) is an architecture pattern that enhances a language model's responses by grounding them in retrieved external knowledge. Rather than relying solely on the parametric knowledge baked into model weights (which can be outdated or hallucinated), RAG dynamically fetches relevant document snippets at inference time and injects them into the LLM's context window.

Key Insight: RAG = Retrieval (find relevant facts) + Generation (formulate a natural-language answer using those facts). The retrieval step acts as a real-time knowledge injection mechanism.

1.2 Why is RAG Needed?

■ Hallucination Reduction

LLMs fabricate plausible-sounding but incorrect facts. Grounding answers in retrieved text reduces this risk significantly.

■ Up-to-date Knowledge

Base LLM weights have a training cutoff. RAG allows the system to query live or recently updated documents.

■ Domain Specialisation

Generic LLMs lack deep domain knowledge. RAG can be pointed at domain-specific PDFs, wikis, or databases.

■ Auditability

Retrieved chunks act as citations, making it possible to trace every answer back to a source document and page number.

1.3 Use Case Overview

A customer support team maintains a set of product manuals, FAQ documents, and policy PDFs. Support agents spend 70% of their time answering questions that are already answered in these documents. The RAG Customer Support Assistant ingests these PDFs, answers Tier-1 queries automatically, and escalates novel or sensitive issues to a human agent via a Human-in-the-Loop (HITL) mechanism — reducing agent workload while maintaining quality and compliance.

2. SYSTEM ARCHITECTURE EXPLANATION

2.1 Two-Phase System

The system operates in two distinct phases: **Ingestion** and **Inference**.

Phase	Trigger	Outcome
Ingestion	New PDF uploaded	Chunks embedded and stored in ChromaDB
Inference	User submits a query	Graph executes; answer returned or escalated

2.2 Component Interactions

During **Ingestion**: PDF → DocumentLoader → ChunkingEngine → EmbeddingService → ChromaDB. The process is a one-time (or on-demand) batch operation. New documents can be added without reprocessing existing ones.

During **Inference**: User Query → LangGraph Input Node → Retriever (queries ChromaDB) → LLM Generator → Router → Output Node (or HITL Node). The graph enforces a strict, auditable processing order via typed state transitions.

2.3 Why LangGraph for Orchestration?

LangGraph models the query lifecycle as a **Directed Acyclic Graph (DAG)** with typed state. Unlike a simple sequential chain, LangGraph allows:

- Conditional branching (answer vs. escalate) without imperative if/else spaghetti.
- First-class HITL support — the graph can pause, await external input, and resume.
- Full state introspection and replay for debugging.
- Easy extension — new nodes (e.g., memory, feedback) can be added without refactoring.

3. DESIGN DECISIONS

■ **Chunk Size: 500 chars, Overlap: 50**

A 500-character chunk is large enough to contain a complete thought or FAQ answer, yet small enough to fit within the LLM context along with 3–4 other chunks and the system prompt. A 50-character overlap prevents key information from being split across chunk boundaries. Empirically, 400–600 character chunks outperform both very small (200) and very large (1000+) chunks on retrieval precision for customer support corpora.

■ **Embedding Strategy: OpenAI text-embedding-3-small**

This model produces 1536-dimensional embeddings at a cost of ~\$0.02 per 1M tokens. It outperforms older ada-002 embeddings on MTEB benchmarks. For organisations with privacy requirements, a local SentenceTransformer (all-MiniLM-L6-v2) provides a cost-free alternative at slightly lower accuracy.

■ **Retrieval: Top-k=4 with Cosine Similarity**

Fetching 4 chunks balances context completeness against LLM token budget. Cosine similarity is chosen over L2 (Euclidean) distance because it is magnitude-invariant — suitable for normalised embedding vectors. k=4 was selected empirically: k<3 misses multi-aspect questions; k>6 dilutes the context with weakly-relevant passages.

■ **Prompt Design: Context-First, Instruction-Guarded**

The system prompt instructs the LLM to answer ONLY from provided context, to state 'I don't have enough information' if context is insufficient, and to never invent facts. This prompt design has been shown to reduce hallucination rates by 40–60% compared to open-ended prompting.

■ **Vector Store: ChromaDB over Pinecone/Weaviate**

ChromaDB runs fully locally with zero infrastructure cost — ideal for an internship prototype and small-to-medium document sets (< 100k chunks). The LangChain integration is one-line. For production at scale, the same interface can be swapped for Pinecone or Weaviate.

4. WORKFLOW EXPLANATION

4.1 LangGraph Usage

The workflow is defined as a **StateGraph** over a **GraphState TypedDict**. Each node is a pure Python function that reads from and writes to the shared state dictionary. Nodes are connected by edges; conditional edges use a routing function to select the next node dynamically. The graph is compiled once at startup and invoked for each user query.

4.2 Node Responsibilities

Node	Responsibility	Key State Mutations
input_node	Classify query intent using keyword rules or a small classifier	Sets: intent
retrieve_node	Embed query; call ChromaDB <code>similarity_search(k=4)</code>	Sets: retrieved, scores
generate_node	Build prompt with context; call LLM; parse answer + confidence	Sets: answer, confidence
route_node	Evaluate routing criteria; return 'output' or 'escalate'	No mutation (decision only)
escalate_node	Create EscalationTicket; enqueue for human review	Sets: escalated=True, ticket_id
hitl_node	Poll/await human response; inject into state	Sets: human_response
output_node	Format final response; log to audit trail	Sets: final_answer

4.3 State Transitions

Every node transition is type-checked against the **GraphState** schema at compile time. If a node writes an unexpected key, LangGraph raises a validation error before the graph runs. This prevents silent data corruption and makes debugging straightforward. The compiled graph is essentially an immutable execution plan — each invocation creates a fresh state snapshot.

5. CONDITIONAL LOGIC

5.1 Intent Detection

The `input_node` classifies each query into one of three intents using a two-stage approach:

- **Rule-based fast path:** Keyword matching for common FAQ patterns (e.g., 'how do I', 'what is', 'when does') → `intent = 'faq'`
- **LLM-based slow path:** For ambiguous queries, a lightweight prompt asks the LLM to classify as 'faq', 'complaint', or 'out_of_scope'.
- **Fallback:** If classification fails, intent defaults to 'faq' to avoid false escalations.

5.2 Routing Decision Matrix

Condition	Decision	Rationale
<code>scores_max >= 0.45 AND conf >= 0.70</code>	Output	High-confidence, context-supported answer
<code>scores_max < 0.45</code>	Escalate	Knowledge gap — question not in knowledge base
<code>conf < 0.70</code>	Escalate	LLM uncertain — risk of wrong answer
<code>intent == 'complaint' + short ans</code>	Escalate	Complaint needs human empathy
<code>'I don't know' in answer</code>	Escalate	LLM self-flagged uncertainty
<code>intent == 'out_of_scope'</code>	Output	Polite refusal message; no escalation

6. HITL IMPLEMENTATION

6.1 Role of Human Intervention

HITL acts as the system's safety valve — it catches queries that fall outside the automation envelope. Rather than returning a wrong or confident-sounding hallucinated answer, the system transparently escalates to a human agent who has access to the full query context and retrieved chunks. The agent's response is injected back into the graph state, formatted by `output_node`, and returned to the user.

6.2 Implementation Detail

```
# escalate_node creates a ticket
def escalate_node(state: GraphState) -> GraphState:
    ticket = hitl_manager.escalate(
        query = state['query'],
        context = state['retrieved'],
        reason = 'low_confidence' if state['confidence'] < 0.7 else 'no_context'
    )
    return {**state, 'escalated': True, 'ticket_id': ticket.ticket_id}

# hitl_node polls until resolved (with timeout)
def hitl_node(state: GraphState) -> GraphState:
    response = hitl_manager.poll_resolved(
        ticket_id = state['ticket_id'],
        timeout_s = 300 # 5-minute SLA
    )
    if response is None:
        response = 'Our team will follow up with you via email within 1 hour.'
    return {**state, 'human_response': response}
```

6.3 Benefits & Limitations

Benefits	Limitations
Prevents wrong answers on sensitive queries	Introduces latency (human response time)
Provides labelled data for model improvement	Requires 24/7 staffed support queue for SLA
Builds user trust — they know a human is involved	Asynchronous UX — user must wait or check back
Audit trail for compliance (e.g., GDPR, HIPAA contexts)	Agent quality varies; inconsistent responses possible

7. CHALLENGES & TRADE-OFFS

■ Retrieval Accuracy vs. Speed

High accuracy: small k , tight similarity threshold → more escalations, slower.

High speed: large k , loose threshold → more noise, lower answer quality.

Resolution: $k=4$ with threshold=0.45 was empirically tuned on a held-out query set to balance precision and recall.

■ Chunk Size vs. Context Quality

Small chunks (200 chars): precise retrieval but may miss surrounding context.

Large chunks (1000 chars): rich context but noisy retrieval and high token cost.

Resolution: 500 chars + 50 overlap balances semantic completeness with retrieval precision.

■ Cost vs. Performance

GPT-4o: highest quality, ~\$0.005/query.

Llama-3 (local): zero marginal cost, ~15% lower answer quality on complex queries.

Resolution: Use GPT-4o for production; Llama-3 for development/cost-capped deployments.

■ Hallucination Risk

Even with context injection, LLMs can mix retrieved facts with parametric knowledge.

Mitigation: Strict prompt instruction ('answer ONLY from the provided context'), confidence scoring, and HITL escalation for low-confidence outputs.

■ Freshness of Knowledge Base

Documents ingested at time T are stale by $T+\delta$. Mitigation: Implement a scheduled re-ingestion pipeline; timestamp each chunk; filter out chunks older than a configurable TTL for time-sensitive queries.

8. TESTING STRATEGY

8.1 Testing Approach

Test Type	What is Tested	Tool / Method
Unit Tests	Individual node functions, chunker, embedder	pytest + mock LLM
Integration Tests	End-to-end graph execution with real ChromaDB	pytest + real APIs
Retrieval Eval	Recall@k and MRR on a held-out query set	Custom eval harness
LLM Eval	Answer quality (BLEU, ROUGE, G-Eval) vs. gold answers	LangSmith / RAGAS
HITL Tests	Ticket creation, human inject, response formatting	pytest + fixture data
Load Tests	Concurrent query throughput, P95 latency	Locust

8.2 Sample Test Queries

Query	Expected Behaviour	Pass Criteria
'What is the return policy?'	Answer from FAQ chunk	confidence >= 0.80
'I want to cancel my order right now!'	HITL escalation	escalated == True
'What is the capital of France?'	Out-of-scope refusal	intent = out_of_scope
'How do I reset my password?'	Step-by-step answer	sources non-empty
[Empty string]	Graceful error message	No exception raised

9. FUTURE ENHANCEMENTS

■ Multi-Document Support

Extend the ingestion pipeline to handle multiple PDFs simultaneously. Implement metadata-based filtering so queries about 'shipping policy' only search the shipping_policy.pdf collection, reducing noise.

■ Feedback Loop

After each answered query, present a thumbs-up/thumbs-down to the user. Negative feedback flags the response for review and feeds into a fine-tuning or RAG re-ranking dataset.

■ Conversation Memory

Add a memory node to LangGraph that injects the last N turns of conversation into the context, enabling multi-turn dialogues ('referring to my earlier question...').

■ Hybrid Retrieval

Combine dense (embedding) retrieval with sparse (BM25 keyword) retrieval using Reciprocal Rank Fusion (RRF). This improves recall for queries with rare or specific terminology.

■ Confidence Fine-tuning

Train a lightweight classifier on logged (query, retrieved_chunks, answer) triples to predict answer confidence more accurately than LLM self-assessment.

■ Production Deployment

Containerise the system with Docker. Deploy LangGraph workflow behind FastAPI on a Kubernetes cluster. Use Pinecone or Weaviate in managed cloud mode. Instrument with OpenTelemetry for distributed tracing.

This document was produced as part of an internship project. All design decisions reflect current best practices as of 2025 and should be validated against updated benchmarks and tooling before production deployment.